Student Information

Name: Colton Zachary

UCO Email: czachary@uco.edu

Project Number: p04

Server Username: xl025

Implementation Discussion

The AVL tree was implemented by completing the missing functions in the provided framework. The rotate_right() function mirrors the behavior of rotate_left() by promoting the left child of a subtree and adjusting the pointers so the original root becomes the right child of the new root. This preserves the binary search tree ordering while correcting height imbalance.

The left_balance() function restores AVL balance when a node becomes left-heavy after an insertion. Two cases are handled: a single right rotation when the left subtree is also left-heavy, and a double rotation (left then right) when the left subtree is right-heavy. Balance factors for all affected nodes are updated accordingly.

For the extra credit portion, the avl_delete() function was implemented using a recursive deletion strategy similar to standard binary search tree deletion. After removing a node, the algorithm checks whether the height of the subtree has decreased and performs the necessary AVL rotations to restore balance.

Program Test Run

The program was compiled using the provided makefile and executed with the input files provided with the assignment. Example commands used:

```
make
./p04 input1.dat
./p04 input2.dat
```

```
(base) coltonzachary@Coltons-MacBook-Air-4 Code % make
./p04 input1.dat
./p04 input2.dat
rm -f *.o
g++ -c -g main.cpp
g++ -c -g utility.cpp
g++ -o p04 main.o utility.o

++++++++++++++++++++++
92:  90  95
90:  70  -
95:  93  110
++++++++++++++++++++++


++++++++++++++++++++++
10:  5  13
5:  3  8
3:  2  4
2:  1  -
8:  7  9
7:  6  -
13:  11  19
11:  -  12
19:  15  25
15:  14  18
25:  20  -
++++++++++++++++++++++
```

Source Code

```
template <class Record>

struct AVL_node: public Binary_node<Record> {

  //additional data member:

  Balance_factor balance;

  //constructors:

  AVL_node();

  AVL_node(const Record &x);



  //overridden virtual functions:

  void set_balance(Balance_factor b);

  Balance_factor get_balance() const;

};




template <class Record>

AVL_node<Record>::AVL_node():Binary_node<Record>(){

  balance = equal_height; // The balance factor is initailized to be
equal_height

}




template <class Record>

AVL_node<Record>::AVL_node(const Record &x):Binary_node<Record>(x){

  balance = equal_height; // The balance factor is initailized to be
equal_height
```

```
}



template <class Record>
void AVL_node<Record>::set_balance(Balance_factor b){
  balance = b;
}


template <class Record>
Balance_factor AVL_node<Record>::get_balance() const{
  return balance;
}



template <class Record>
class AVL_tree: public Search_tree<Record> {
public:
  Error_code insert(const Record &new_data);
  Error_code remove(const Record &old_data);
protected:
  //  Auxiliary functions
  Error_code avl_insert(Binary_node<Record>* &sub_root, const Record
&new_data, bool &taller);
  Error_code avl_delete(Binary_node<Record>* &sub_root, const Record
&target, bool &shorter);
```

```
  void left_balance(Binary_node<Record>* &sub_root);

  void right_balance(Binary_node<Record>* &sub_root);

  void rotate_left(Binary_node<Record>* &sub_root);

  void rotate_right(Binary_node<Record>* &sub_root);

};


template <class Record>

Error_code AVL_tree<Record>::insert(const Record &new_data)

/*

Post: If the key of new_data is already in the AVL_tree, a code

     of duplicate_error is returned.

     Otherwise, a code of success is returned and the Record new_data

     is inserted into the tree in such a way that the properties of

     an AVL tree are preserved.

Uses: avl_insert.

*/

{

  bool taller;

  return avl_insert(this->root, new_data, taller);

}


template <class Record>

Error_code AVL_tree<Record>::remove(const Record &target)

/*

Post: If a Record with a key matching that of target belongs to the
```

```
     AVL_tree, a code of success is returned, and the corresponding node
     is removed from the tree.  Otherwise, a code of not_present is
returned.
Uses: Function search_and_destroy
*/
{
  bool shorter;
  return avl_delete(this->root, target, shorter);
}


template <class Record>
Error_code AVL_tree<Record>::avl_insert(Binary_node<Record>* &sub_root,
          const Record &new_data, bool &taller)
/*
Pre:  sub_root is either NULL or points to a subtree of the AVL_tree
Post: If the key of new_data is already in the subtree, a
     code of duplicate_error is returned.
     Otherwise, a code of success is returned and the Record new_data
     is inserted into the subtree in such a way that the
     properties of an AVL tree have been preserved.
     If the subtree is increased in height, the parameter taller is set to
     true; otherwise it is set to false.
Uses: Methods of struct AVL_node; functions avl_insert
     recursively, left_balance, and right_balance.
*/
```

```
{
  Error_code result = success;
  if (sub_root == NULL) {
     sub_root = new AVL_node<Record>(new_data);
     taller = true;  //taller is true for every new node creation
  }
  else if (new_data == sub_root->data) {
     result = duplicate_error;
     taller = false;
  }
  //insert to LST
  else if (new_data < sub_root->data) {
     result = avl_insert(sub_root->left, new_data, taller);
     if (taller == true)
     switch (sub_root->get_balance()) {
        case left_higher: //lh before insertion, now unbalanced
           left_balance(sub_root);
           taller = false;
           break;
      case equal_height:
           sub_root->set_balance(left_higher);
           break;
        case right_higher:
           sub_root->set_balance(equal_height);
           taller = false;
```

```
            break;
        }
  }
  //insert to RST
  else {
     result = avl_insert(sub_root->right, new_data, taller);
     if (taller == true)
        switch (sub_root->get_balance()) {
        case left_higher:
           sub_root->set_balance(equal_height);
           taller = false;
           break;
        case equal_height:
           sub_root->set_balance(right_higher);
           break;
        case right_higher:
           right_balance(sub_root);
           taller = false;
           break;
        }
  }
  return result;
}


template <class Record>
```

```
void AVL_tree<Record>::right_balance(Binary_node<Record> *&sub_root)
/*
Pre:  sub_root points to a subtree of an AVL_tree that
     is unbalanced on the right.
Post: The AVL properties have been restored to the subtree.
*/
{
  Binary_node<Record>* &right_tree = sub_root->right;


  switch (right_tree->get_balance()) {
  // case right_higher: sigle left rotation
  // O  ub --> subroot
  //  \
  //   O  rh  --> right_tree
  //    \
  //     O
  case right_higher:  //  single left rotation
     sub_root->set_balance(equal_height);
     right_tree->set_balance(equal_height);
     rotate_left(sub_root); //pointer adjustment
     break;
  case equal_height:  //  impossible case
     cout << "WARNING: If you see this in an insertion, program error is
detected in right_balance" << endl;
     break;
```

```
// case left_higher: double rotation left
// O ub --> sub_root
//  \
//   O lh --> right_tree
//  /
// O three cases --> sub_tree
case left_higher:
   Binary_node<Record> *sub_tree = right_tree->left;
  //set balance of sub_root and right_tree assuming rotation is done
   switch (sub_tree->get_balance()) {
   case equal_height:
      sub_root->set_balance(equal_height);
      right_tree->set_balance(equal_height);
      break;
   case left_higher:
      sub_root->set_balance(equal_height);
      right_tree->set_balance(right_higher);
      break;
   case right_higher:
      sub_root->set_balance(left_higher);
      right_tree->set_balance(equal_height);
      break;
   }
  //set balance of sub_tree after rotation
   sub_tree->set_balance(equal_height);
```

```
    //perform actual rotation
     rotate_right(right_tree);
     rotate_left(sub_root);
     break;
  }
}


template <class Record>
void AVL_tree<Record>::rotate_left(Binary_node<Record> *&sub_root)
/*
Pre:  sub_root points to a subtree of the AVL_tree.
     This subtree has a nonempty right subtree.
Post: sub_root is reset to point to its former right child, and the former
     sub_root node is the left child of the new sub_root node.
*/
{
  if (sub_root == NULL || sub_root->right == NULL)     //  impossible
cases
     cout << "WARNING: program error detected in rotate_left" << endl;
  else {
     Binary_node<Record> *right_tree = sub_root->right;
     sub_root->right = right_tree->left;
     right_tree->left = sub_root;
     sub_root = right_tree;
  }
```

```
}


template <class Record>
void AVL_tree<Record>::rotate_right(Binary_node<Record> *&sub_root)
/*
Pre:  sub_root points to a subtree of the AVL_tree.
     This subtree has a nonempty left subtree.
Post: sub_root is reset to point to its former left child, and the former
     sub_root node is the right child of the new sub_root node.
*/
{
  // TODO 1
  if (sub_root == NULL || sub_root->left == NULL)     //  impossible cases
     cout << "WARNING: Program error detected in rotate_right" << endl;
  else {
     Binary_node<Record> *left_tree = sub_root->left; // pointer to left
child of sub_root
     sub_root->left = left_tree->right; // left child of sub_root becomes
right child of left_tree
     left_tree->right = sub_root; // sub_root becomes right child of
left_tree
     sub_root = left_tree; // sub_root now points to left_tree, which is
the new root of the subtree
  }
```

```
}


template <class Record>
void AVL_tree<Record>::left_balance(Binary_node<Record>* &sub_root)
/*
Pre:  sub_root points to a subtree of an AVL_tree that
     is doubly unbalanced on the left.
Post: The AVL properties have been restored to the subtree.
*/
{
  // TODO 2
  Binary_node<Record>* &left_tree = sub_root->left; // pointer to left
child of sub_root

  switch (left_tree->get_balance()) { // check balance of left child of
sub_root

  // single right rotation
  case left_higher:
     sub_root->set_balance(equal_height);
     left_tree->set_balance(equal_height);
     rotate_right(sub_root);
     break;

  case equal_height:
     cout << "WARNING: Program error is detected in left_balance" << endl;
```

```
    break;


  // double rotation
  case right_higher:
    Binary_node<Record>* sub_tree = left_tree->right;


    switch (sub_tree->get_balance()) { // check balance of left_tree's
right child
    case equal_height: // sub_tree is the new root of the subtree after
rotation, so its balance will be equal_height
        sub_root->set_balance(equal_height); // sub_root becomes child of
sub_tree, so its balance will be equal_height
        left_tree->set_balance(equal_height); // left_tree becomes child
of sub_tree, so its balance will be equal_height
        break;


    case left_higher: // sub_tree is left child of left_tree, so
left_tree becomes right_higher and sub_root becomes left_higher
        sub_root->set_balance(right_higher);
        left_tree->set_balance(equal_height);
        break;


    case right_higher: // sub_tree is right child of left_tree, so
left_tree becomes equal_height and sub_root becomes left_higher
        sub_root->set_balance(equal_height);
        left_tree->set_balance(left_higher);
        break;
```

```
    }

    sub_tree->set_balance(equal_height);

    rotate_left(left_tree);
    rotate_right(sub_root);
    break;
  }

}

template <class Record>
Error_code AVL_tree<Record>::avl_delete(Binary_node<Record>* &sub_root,
          const Record &target, bool &shorter)
{
  Error_code result = success;

  // TODO 3: bonus task
  if (sub_root == NULL) { // target not found
    shorter = false;
    return not_present;
  }

  // search left
```

```
  if (target < sub_root->data) { // target is in left subtree

     result = avl_delete(sub_root->left, target, shorter); // recursively
delete target from left subtree

     if (shorter == true) { // left subtree is shorter after deletion,
check balance of sub_root

        switch (sub_root->get_balance()) { // check balance of sub_root

        case left_higher: // left subtree was higher before deletion, now
it is balanced

           sub_root->set_balance(equal_height);

           shorter = true;

           break;


        case equal_height: // left subtree was balanced before deletion,
now it is higher

           sub_root->set_balance(right_higher);

           shorter = false;

           break;


        case right_higher: { // left subtree was lower before deletion,
now it is unbalanced

           Binary_node<Record>* &right_tree = sub_root->right;


           switch (right_tree->get_balance()) { // check balance of right
child of sub_root

           case right_higher: // right subtree is higher, so after
rotation, sub_root becomes balanced and right_tree becomes balanced

              sub_root->set_balance(equal_height);
```

```
            right_tree->set_balance(equal_height);
            rotate_left(sub_root);
            shorter = true;
            break;

         case equal_height: // right subtree is balanced, so after
rotation, sub_root becomes right_higher and right_tree becomes left_higher
            sub_root->set_balance(right_higher);
            right_tree->set_balance(left_higher);
            rotate_left(sub_root);
            shorter = false;
            break;

         case left_higher: {
            Binary_node<Record>* sub_tree = right_tree->left;

            switch (sub_tree->get_balance()) { // check balance of
right_tree's left child
            case equal_height:
               sub_root->set_balance(equal_height);
               right_tree->set_balance(equal_height);
               break;
            case left_higher:
               sub_root->set_balance(equal_height);
               right_tree->set_balance(right_higher);
               break;
```

```
            case right_higher:
               sub_root->set_balance(left_higher);
               right_tree->set_balance(equal_height);
               break;
            }

            sub_tree->set_balance(equal_height);
            rotate_right(right_tree);
            rotate_left(sub_root);
            shorter = true;
            break;
         }
         }
         break;
      }
      }
   }
}

// search right
else if (target > sub_root->data) {// target is in right subtree
   result = avl_delete(sub_root->right, target, shorter);

   if (shorter == true) {
      switch (sub_root->get_balance()) {
```

```
        case right_higher:
            sub_root->set_balance(equal_height);
            shorter = true;
            break;

        case equal_height:
            sub_root->set_balance(left_higher);
            shorter = false;
            break;

        case left_higher: {
            Binary_node<Record>* &left_tree = sub_root->left;

            switch (left_tree->get_balance()) {
            case left_higher:
                sub_root->set_balance(equal_height);
                left_tree->set_balance(equal_height);
                rotate_right(sub_root);
                shorter = true;
                break;

            case equal_height:
                sub_root->set_balance(left_higher);
                left_tree->set_balance(right_higher);
                rotate_right(sub_root);
```

```
            shorter = false;
            break;

        case right_higher: {
            Binary_node<Record>* sub_tree = left_tree->right;

            switch (sub_tree->get_balance()) {
            case equal_height:
                sub_root->set_balance(equal_height);
                left_tree->set_balance(equal_height);
                break;
            case left_higher:
                sub_root->set_balance(right_higher);
                left_tree->set_balance(equal_height);
                break;
            case right_higher:
                sub_root->set_balance(equal_height);
                left_tree->set_balance(left_higher);
                break;
            }

            sub_tree->set_balance(equal_height);
            rotate_left(left_tree);
            rotate_right(sub_root);
            shorter = true;
```

```
            break;
        }
        }
        break;
    }
    }
  }
}

// found target
else { // target is in current node
  Binary_node<Record>* to_delete = sub_root;

  // no right child
  if (sub_root->right == NULL) {
    sub_root = sub_root->left;
    delete to_delete;
    to_delete = nullptr;
    shorter = true;
  }

  // no left child
  else if (sub_root->left == NULL) {
    sub_root = sub_root->right;
    delete to_delete;
```

```
        to_delete = nullptr;
        shorter = true;
    }

    // two children
    else {
        Binary_node<Record>* predecessor = sub_root->left;
        while (predecessor->right != NULL) {
            predecessor = predecessor->right;
        }

        sub_root->data = predecessor->data;
        result = avl_delete(sub_root->left, predecessor->data, shorter);

        if (shorter == true) {
            switch (sub_root->get_balance()) {
            case left_higher:
                sub_root->set_balance(equal_height);
                shorter = true;
                break;

            case equal_height:
                sub_root->set_balance(right_higher);
                shorter = false;
                break;
```

```
        case right_higher: {
            Binary_node<Record>* &right_tree = sub_root->right;

            switch (right_tree->get_balance()) {
            case right_higher:
                sub_root->set_balance(equal_height);
                right_tree->set_balance(equal_height);
                rotate_left(sub_root);
                shorter = true;
                break;

            case equal_height:
                sub_root->set_balance(right_higher);
                right_tree->set_balance(left_higher);
                rotate_left(sub_root);
                shorter = false;
                break;

            case left_higher: {
                Binary_node<Record>* sub_tree = right_tree->left;

                switch (sub_tree->get_balance()) {
                case equal_height:
                    sub_root->set_balance(equal_height);
```

```
                right_tree->set_balance(equal_height);
                break;
            case left_higher:
                sub_root->set_balance(equal_height);
                right_tree->set_balance(right_higher);
                break;
            case right_higher:
                sub_root->set_balance(left_higher);
                right_tree->set_balance(equal_height);
                break;
            }

            sub_tree->set_balance(equal_height);
            rotate_right(right_tree);
            rotate_left(sub_root);
            shorter = true;
            break;
        }
        }
        break;
    }
    }
  }
 }
}
```

```
  return result;
}
```

```
enum Balance_factor { left_higher, equal_height, right_higher };

template <class Entry>
struct Binary_node{
//    data members:
  Entry data;
  Binary_node<Entry> *left;
  Binary_node<Entry> *right;
//    constructors:
  Binary_node();
  Binary_node(const Entry &x);

  // virtual methods:
  virtual void set_balance(Balance_factor b);
  virtual Balance_factor get_balance() const;
};

template <class Entry>
Binary_node<Entry>::Binary_node(){
```

```
  left = NULL;

  right = NULL;

}


template <class Entry>

Binary_node<Entry>::Binary_node(const Entry &x){

  data = x;

  left = NULL;

  right = NULL;

}


// Dummy functions

template <class Entry>

void Binary_node<Entry>::set_balance(Balance_factor b){

}


template <class Entry>

Balance_factor Binary_node<Entry>::get_balance() const {

  return equal_height;

}
```

```
template <class Entry>

class Binary_tree {

public:
```

```
  Binary_tree();

  bool empty() const;

  void preorder(void (*visit)(Entry &));

  void inorder(void (*visit)(Entry &));

  void postorder(void (*visit)(Entry &));


  int size() const;

  void clear();

  int height() const;


  void print() const; // A method to print out the tree using preorder


  Binary_tree(const Binary_tree<Entry> &original);

  Binary_tree<Entry>& operator =(const Binary_tree<Entry> &original);

  ~Binary_tree();

protected:

  // Auxiliary functions

  void recursive_preorder(Binary_node<Entry> *sub_root, void
(*visit)(Entry &));

  void recursive_inorder(Binary_node<Entry> *sub_root, void (*visit)(Entry
&));

  void recursive_postorder(Binary_node<Entry> *sub_root, void
(*visit)(Entry &));

  int recursive_size(Binary_node<Entry> *sub_root) const;

  void recursive_clear(Binary_node<Entry> *sub_root);

  int recursive_height(Binary_node<Entry> *sub_root) const;
```

```
  Binary_node<Entry>* recursive_copy(Binary_node<Entry> *sub_root);


  void recursive_preorder_print_node(Binary_node<Entry> *sub_root) const;


  // Data member

  Binary_node<Entry>* root;

};


template <class Entry>

Binary_tree<Entry>::Binary_tree()

/*

Post: An empty binary tree has been created.

*/

{

  root = NULL;

}


template <class Entry>

Binary_tree<Entry>::Binary_tree(const Binary_tree<Entry> &original)

/*

Post: A new binary tree has been created with the same structure and
content as those of original.

Uses: The function recursive_copy

*/

{
```

```
 root = recursive_copy(original.root);

}


template <class Entry>

Binary_tree<Entry>& Binary_tree<Entry>::operator =(const
Binary_tree<Entry> &original)

/*

Post: The binary tree is assigned to copy a parameter

*/

{

  Binary_tree<Entry> new_tree(original);

  Binary_node<Entry>* temp_root = root;

  root = new_tree.root;

  new_tree.root = temp_root; // new tree root and its subtrees will be
deleted by its destructor when this method returns

  return *this;

}


template <class Entry>

Binary_tree<Entry>::~Binary_tree() {

  clear();

}


template <class Entry>

bool Binary_tree<Entry>::empty() const

/*
```

```
Post: A result of true is returned if the binary tree is empty.
     Otherwise, false is returned.
*/
{
  return root == NULL;
}

template <class Entry>
void Binary_tree<Entry>::preorder(void (*visit)(Entry &))
/*
Post: The tree has been traversed in preorder sequence.
Uses: The function recursive_preorder
*/
{
  recursive_preorder(root, visit);
}

template <class Entry>
void Binary_tree<Entry>::inorder(void (*visit)(Entry &))
/*
Post: The tree has been traversed in inorder sequence.
Uses: The function recursive_inorder
*/
{
  recursive_inorder(root, visit);
```

```
}


template <class Entry>
void Binary_tree<Entry>::postorder(void (*visit)(Entry &))
/*
Post: The tree has been traversed in postorder sequence.
Uses: The function recursive_postorder
*/
{
  recursive_postorder(root, visit);
}


template <class Entry>
int Binary_tree<Entry>::size() const
/*
Post: The number of entries in the binary tree is returned.
Uses: The function recursive_size
*/
{
  return recursive_size(root);
}



template <class Entry>
void Binary_tree<Entry>::clear()
```

```
/*
Post: Dispose of all the nodes of the binary tree.
Uses: The function recursive_clear.
*/
{
  recursive_clear(root);
  root = NULL;
}


template <class Entry>
int Binary_tree<Entry>::height() const
/*
Post: The height of the binary tree is returned.
Uses: The function recursive_height
*/
{
  return recursive_height(root);
}



template <class Entry>
void Binary_tree<Entry>::print() const{
/*
Post: The tree has been traversed in preorder.  Each node and its two
children is printed
```

```
Uses: The function recursive_preorder_print_node
*/
  cout << endl;
  cout << "++++++++++++++++++++++" << endl;
  if(root == NULL)
     cout << "EMPTY TREE" << endl;
  else{
     if(root->left == NULL && root->right == NULL)
        cout << root->data << ":  -  -" << endl;

     recursive_preorder_print_node(root);
  }
  cout << "++++++++++++++++++++++" << endl;
  cout << endl;
}

// Functions

template <class Entry>
void Binary_tree<Entry>::recursive_preorder(Binary_node<Entry> *sub_root,
                                            void (*visit)(Entry &))
/*
Pre:  sub_root is either NULL or points to a subtree of the Binary_tree.
Post: The subtree has been traversed in preorder sequence.
Uses: The function recursive_preorder recursively
```

```
*/

{

  if (sub_root != NULL) {

     (*visit)(sub_root->data);

     recursive_preorder(sub_root->left, visit);

     recursive_preorder(sub_root->right, visit);

  }

}


template <class Entry>

void Binary_tree<Entry>::recursive_inorder(Binary_node<Entry> *sub_root,

                                           void (*visit)(Entry &))

/*

Pre:  sub_root is either NULL or points to a subtree of the Binary_tree.

Post: The subtree has been traversed in inorder sequence.

Uses: The function recursive_inorder recursively

*/

{

  if (sub_root != NULL) {

     recursive_inorder(sub_root->left, visit);

     (*visit)(sub_root->data);

     recursive_inorder(sub_root->right, visit);

  }

}
```

```
template <class Entry>
void Binary_tree<Entry>::recursive_postorder(Binary_node<Entry> *sub_root,
                                             void (*visit)(Entry &))
/*
Pre:  sub_root is either NULL or points to a subtree of the Binary_tree.
Post: The subtree has been traversed in postorder sequence.
Uses: The function recursive_postorder recursively
*/
{
  if (sub_root != NULL) {
     recursive_postorder(sub_root->left, visit);
     recursive_postorder(sub_root->right, visit);
     (*visit)(sub_root->data);
  }
}


template <class Entry>
int Binary_tree<Entry>::recursive_size(Binary_node<Entry> *sub_root) const
/*
Pre:  sub_root is either NULL or points to a subtree of the Binary_tree.
Post: The number of entries in the subtree is returned.
Uses: The function recursive_size recursively
*/
{
  if (sub_root == NULL)
```

```
    return 0;
  else
    return 1 + recursive_size(sub_root->left) +
recursive_size(sub_root->right);
}


template <class Entry>
void Binary_tree<Entry>::recursive_clear(Binary_node<Entry> *sub_root)
/*
Pre:  sub_root is either NULL or points to a subtree of the Binary_tree.
Post: All the nodes in the subtree are disposed of in postorder.
Uses: The function recursive_clear recursively
*/
{
  if (sub_root != NULL) {
    recursive_clear(sub_root->left);
    recursive_clear(sub_root->right);
    // Note that at this moment, sub_root->left and sub_root->right may
be dangled pointers.
    delete sub_root;
  }
}


template <class Entry>
int Binary_tree<Entry>::recursive_height(Binary_node<Entry> *sub_root)
const
```

```
/*

Pre:  sub_root is either NULL or points to a subtree of the Binary_tree.

Post: the height of the subtree is returned.

Uses: The function recursive_height recursively

*/

{

  if (sub_root == NULL)

     return 0;

  else {

     int left_height = recursive_height(sub_root->left);

     int right_height = recursive_height(sub_root->right);

     if (left_height >= right_height)

        return 1 + left_height;

     else

        return 1 + right_height;

  }

}


template <class Entry>

Binary_node<Entry>* Binary_tree<Entry>::recursive_copy(Binary_node<Entry>*
sub_root)

/*

Pre:  sub_root is either NULL or points to a subtree of the Binary_tree.

Post: returns a pointer to the root of a new binary tree that has exactly
the same structure and content as those of the subtree.

Uses: The function recursive_copy recursively
```

```
*/

{

  if (sub_root == NULL)

     return NULL;

  else {

     Binary_node<Entry>* new_sub_root = new
Binary_node<Entry>(sub_root->data);

     new_sub_root->left = recursive_copy(sub_root->left);

     new_sub_root->right = recursive_copy(sub_root->right);

     return new_sub_root;

  }

}




template <class Entry>

void Binary_tree<Entry>::recursive_preorder_print_node(Binary_node<Entry>
*sub_root) const

/*

Pre:  sub_root is either NULL or points to a subtree of the Binary_tree.

Post: The subtree has been traversed in preorder sequence.

Uses: The function recursive_preorder_print_node recursively

*/

{

  if (sub_root != NULL) {

     if(sub_root->left != NULL || sub_root->right != NULL){

        cout << sub_root->data << ":  ";
```

```
        if(sub_root->left != NULL)
           cout << sub_root->left->data << "  ";
        else
           cout << "-  "; // "-" represents no child
        if(sub_root->right != NULL)
           cout << sub_root->right->data << " ";
        else
           cout << "-  "; // "-" represents no child
        //cout << "; ";
        cout << endl;
     }
     recursive_preorder_print_node(sub_root->left);
     recursive_preorder_print_node(sub_root->right);
  }
}



#include "utility.h"
#include "Binary_node.h"
#include "Binary_tree.h"
#include "Search_tree.h"
#include "AVL_node.h"
#include "AVL_tree.h"

/*
```

```
* Handle command line error
*/
struct CommandLineException
{
  CommandLineException()
  {
     cout << endl;
     cout << "Incorrect command line arguments." << endl;
     cout << "The path of input file is incorrect." << endl;
     cout << "An example to execute this program:" << endl;
     cout << "    ./p04 input.dat " << endl;
  }

};

/*
* Handle exception when openning a file
*/
struct FileException {
  FileException(const string file) {
   cout << endl << "'" << file << "'" << " could not be opened." << endl;
  }
};

/*
```

```
* Handle unexpected file content
*/
struct FileContentException {
  FileContentException() {
     cout << endl << "The content of the file doesn't match the
requirements." << endl;
     cout << "This file can't be further processed." << endl;
  }
};


int main(int argc, char** argv){
  try {
     // handle command line input
     if (argc != 2) {
        throw CommandLineException();
     }

     ifstream input_file;
     input_file.open(argv[1]);
     if (!input_file) {
        throw FileException(argv[1]);
     }

     AVL_tree<int> avl_tree;
     string line;
```

```
    while (input_file >> line) {
        int option = 0;
        if (!line.compare("insert")) {
            option = 1;
        }
        else if (!line.compare("delete")) {
            option = 2;
        }

        input_file >> line;
        if (isNumber(line)) {
            int value = string_to_int(line);
            if (option == 1) {
                avl_tree.insert(value);
            }
            else if (option == 2) {
                avl_tree.remove(value);
            }
        }
        else {
            throw FileContentException();
        }
    }

avl_tree.print();
```

```
  } catch (...) {
     cout << "Program ended unexpectedly." << endl;
     exit(EXIT_FAILURE);
  }

  return 0;
}
```

```
all: p04
p04: removal main.o utility.o
   g++ -o p04 main.o utility.o
main.o: main.cpp
   g++ -c -g main.cpp
utility.o: utility.cpp
   g++ -c -g utility.cpp
removal:
   rm -f *.o


template <class Record>
class Search_tree: public Binary_tree<Record> {
public:
  Error_code insert(const Record &new_data);
  Error_code remove(const Record &old_data);
  Error_code tree_search(Record &target) const;
```

```
protected:

  // Auxiliary functions

  Error_code search_and_insert(Binary_node<Record>* &sub_root, const
Record &new_data);

  Error_code search_and_delete(Binary_node<Record>* &sub_root, const
Record &target);

  Binary_node<Record>* search_for_node(Binary_node<Record>* sub_root,
const Record &target) const;

};




template <class Record>

Error_code Search_tree<Record>::insert(const Record &new_data)

{

  return search_and_insert(this->root, new_data);

}




template <class Record>

Error_code Search_tree<Record>::search_and_insert(

          Binary_node<Record> *&sub_root, const Record &new_data)

{

  if (sub_root == NULL) {

     sub_root = new Binary_node<Record>(new_data);

     return success;

  }
```

```
  else if (new_data < sub_root->data)
     return search_and_insert(sub_root->left, new_data);
  else if (new_data > sub_root->data)
     return search_and_insert(sub_root->right, new_data);
  else return duplicate_error;
}



template <class Record>
Error_code Search_tree<Record>::remove(const Record &target)
/*
Post: If a Record with a key matching that of target belongs to the
     Search_tree, a code of success is returned, and the corresponding
node
     is removed from the tree.  Otherwise, a code of not_present is
returned.
Uses: Function search_and_destroy
*/
{
  return search_and_delete(this->root, target);
}



template <class Record>
Error_code Search_tree<Record>::search_and_delete(
          Binary_node<Record>* &sub_root, const Record &target)
```

```
/*
Pre:  sub_root is either NULL or points to a subtree of the Search_tree.
Post: If the key of target is not in the subtree, a code of not_present
     is returned.  Otherwise, a code of success is returned and the
subtree
     node containing target has been removed in such a way that
     the properties of a binary search tree have been preserved.
Uses: Functions search_and_delete recursively
*/
{
  if (sub_root == NULL)
     return not_present;
  else if (sub_root->data == target) {
     if (sub_root->right == NULL) { // No right child
        Binary_node<Record> *to_delete = sub_root;  //  Remember node to
delete at end.
        sub_root = sub_root->left;
        delete to_delete;
        to_delete = nullptr;
     } else if (sub_root->left == NULL) { // No left child
        Binary_node<Record> *to_delete = sub_root;  //  Remember node to
delete at end.
        sub_root = sub_root->right;
        delete to_delete;
        to_delete = nullptr;
     } else { // subroot has two children
```

```
        // search for the immediate predecessor
        Binary_node<Record> *predecessor_node = sub_root->left;
        while (predecessor_node->right != NULL) {
           predecessor_node = predecessor_node->right;
        }
        // replace the target with the immediate predecessor
        sub_root->data = predecessor_node->data;
        // delete the redundant immediate predecessor
        search_and_delete(sub_root->left, sub_root->data);
     }
  } else if (target < sub_root->data)
     return search_and_delete(sub_root->left, target);
  else
     return search_and_delete(sub_root->right, target);
  return success;
}



template <class Record>
Error_code Search_tree<Record>::tree_search(Record &target) const
/*
Post: If there is an entry in the tree whose key matches that in target,
     the parameter target is replaced by the corresponding record from
     the tree and a code of success is returned.  Otherwise
     a code of not_present is returned.
```

```
Uses: function search_for_node

*/

{

  Error_code result = success;

  Binary_node<Record> *found = search_for_node(this->root, target);

  if (found == NULL)

     result = not_present;

  else

     target = found->data;

  return result;

}




template <class Record>

Binary_node<Record> *Search_tree<Record>::search_for_node(

           Binary_node<Record>* sub_root, const Record &target) const

{

  if (sub_root == NULL || sub_root->data == target) return sub_root;

  else if (sub_root->data < target)

     return search_for_node(sub_root->right, target);

  else return search_for_node(sub_root->left, target);

}
```

```
#include "utility.h"


int string_to_int(const string& s){
  istringstream instr(s);
  int n;
  instr >> n;
  return n;
}


bool isNumber(const string& str)
{
   for (char const &c : str) {
       if (std::isdigit(c) == 0) return false;
   }
   return !str.empty();
}
```

```
#include <iostream>
#include <string>
#include <sstream>
#include <fstream>


using namespace std;


enum Error_code { duplicate_error, not_present, success };
```

```
int string_to_int(const string& s);

bool isNumber(const string& str);
```